

PORTADA

Calidad de Software

Semana 13 — Frameworks de automatización

Universidad de Antioquía · Ingeniería de Sistemas



REPASO RELÁMPAGO

Lo que ya tenemos de la semana 12

- Un **pipeline de CI** encadena build, test, cobertura y análisis estático — si una etapa falla, las siguientes no corren.
- **GitHub Actions** dispara ese pipeline solo, en cada `push` o `pull request` .
- Un pipeline verde es necesario, pero no suficiente: solo verifica lo que alguien pensó en automatizar.



APERTURA

Una prueba de regresión manual, ejecutada a mano, cada semana, durante años

Así prueban muchos equipos que nunca automatizaron su suite E2E — hasta que el sistema crece demasiado para sostenerlo.

TRANSICIÓN

El pipeline necesita algo que ejecutar

La semana 12 automatizó **cuándo** corre la suite. Hoy resolvemos **con qué se construye** esa suite cuando el nivel a probar ya no es una función aislada (semana 10), sino la aplicación completa, a través de su interfaz o su API.



CONCEPTO

Qué es un framework de automatización de pruebas

Es una biblioteca o herramienta que permite **escribir código que controla una aplicación** —un navegador, una API— para ejecutar los mismos pasos que seguiría una persona probando a mano, pero de forma repetible y sin intervención humana.

Corresponde al nivel más alto de la **pirámide de Cohn** (semana 8): pocas pruebas, pero de **extremo a extremo (E2E)** — cubren el sistema completo, incluida la integración entre sus partes.



CONCEPTO · HERRAMIENTA

Selenium

El framework más antiguo y extendido para automatizar navegadores (2004). Implementa **WebDriver**, un protocolo estándar que le habla directamente al navegador —Chrome, Firefox, Edge— para simular clics, escritura y navegación.

- **Ventaja:** soporta prácticamente cualquier lenguaje (Java, Python, C#, JavaScript) y cualquier navegador.
- **Costo:** más verboso que las alternativas modernas — hay que esperar manualmente a que los elementos aparezcan (*waits* explícitos), o la prueba falla de forma intermitente.



CONCEPTO · HERRAMIENTA

Playwright

Framework de Microsoft (2020), diseñado explícitamente para resolver los dolores de Selenium. Su diferencia central: **auto-espera (auto-wait)** — antes de interactuar con un elemento, espera sola a que esté visible y listo, sin que el código lo pida explícitamente.

- Controla Chromium, Firefox y WebKit con la misma API.
- Incluye herramientas propias de depuración: grabación de pruebas, captura de video y *trace viewer* para ver paso a paso por qué falló una ejecución.



CONCEPTO · HERRAMIENTA

Cypress

Framework centrado solo en JavaScript/TypeScript, con una diferencia arquitectónica importante: **corre dentro del propio navegador**, junto a la aplicación, en vez de controlarlo desde afuera como Selenium o Playwright.

- Eso le da acceso directo al DOM y a las peticiones de red de la aplicación, con recarga en vivo mientras se escribe la prueba.
- **Límite conocido:** por su arquitectura, no soporta múltiples pestañas ni dominios distintos con la misma facilidad que Selenium o Playwright.



CONCEPTO · HERRAMIENTA

REST Assured

Los tres frameworks anteriores automatizan la **interfaz**. REST Assured automatiza otra capa: llama directamente a una **API REST** —sin pasar por ninguna pantalla— y valida la respuesta: código de estado, estructura del JSON, tiempos.

Es una biblioteca de Java, pensada para integrarse con JUnit (semana 10). Al saltarse la interfaz, sus pruebas son más rápidas y más estables que las de UI — pero no verifican que lo que el usuario ve en pantalla sea correcto.



CONCEPTO · SÍNTESIS

Los cuatro, uno al lado del otro

Framework	Capa que prueba	Lenguaje(s)	Particularidad
Selenium	UI (navegador)	Cualquiera	El más soportado, el más verboso
Playwright	UI (navegador)	JS/TS, Python, Java, .NET	Auto-espera, depuración incluida
Cypress	UI (navegador)	JS/TS	Corre dentro del navegador
REST Assured	API	Java	No usa interfaz, más rápido y estable

CONCEPTO

Selección según contexto: qué preguntar antes de elegir

- **¿Qué capa hay que probar?** Si es API pura, REST Assured (u otra biblioteca de API testing) gana por velocidad y estabilidad — no tiene sentido pasar por la UI para verificar un contrato de datos.
- **¿Qué lenguaje domina el equipo?** Cypress exige JavaScript; Selenium y Playwright dan flexibilidad si el equipo ya trabaja en Java o Python.
- **¿Cuántos navegadores hay que cubrir?** Selenium y Playwright cubren más combinaciones que Cypress.
- **¿Qué tan nuevo es el proyecto?** Sin código heredado en Selenium, Playwright suele ser hoy la opción con menos fricción para empezar.

EJEMPLO

Un primer test E2E con Playwright

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch()
    page = browser.new_page()
    page.goto("https://the-internet.herokuapp.com/login")
    page.fill("#username", "tomsmith")
    page.fill("#password", "SuperSecretPassword!")
    page.click("button[type='submit']")
    assert "You logged into a secure area" in page.content()
    browser.close()
```

Cinco pasos: abrir el navegador, navegar, llenar dos campos, hacer clic y verificar el resultado. Ni un solo *wait* explícito — Playwright espera solo a que cada elemento esté listo.

TRANSICIÓN

De un script suelto a una suite mantenible

El ejemplo anterior funciona, pero si el `id` de un campo cambia, hay que corregirlo en cada prueba que lo use. La semana 14 resuelve exactamente ese problema con el patrón **Page Object**. Hoy el objetivo es más simple: que el primer test corra.



INTERACCIÓN

Ejercicio rápido: ¿qué framework usarían?

5 minutos, respondan en el chat:

1. Un equipo prueba solo los endpoints de una API de pagos, sin interfaz visual todavía.
2. Un equipo JavaScript necesita pruebas rápidas de un solo dominio, sin pestañas múltiples.
3. Un equipo Java necesita cubrir Chrome, Firefox y Safari por igual.

CONCEPTO EN DISPUTA

¿Más automatización de UI siempre es mejor?

Es tentador leer "automatizar más" como progreso automático. Pero la pirámide de Cohn (semana 8) advierte lo contrario: las pruebas E2E son **lentas, costosas de mantener y frágiles** ante cualquier cambio de interfaz — un cambio de diseño puede romper decenas de pruebas que no tienen nada que ver con la lógica de negocio.

La discusión real de la industria no es "automatizar o no", sino en qué nivel. Muchos equipos maduros invierten deliberadamente más en pruebas unitarias y de API, y reservan la UI para los flujos críticos que de verdad necesitan verse funcionando de punta a punta.



CONCEPTO

Definición pendiente: evidencia de ejecución

El taller de hoy pide **evidencia de ejecución**: no basta con que el código de la prueba exista, hay que demostrar que corrió y qué resultado dio.

- Una **captura de pantalla** de la consola o terminal mostrando el test en verde.
- O un **reporte HTML** que el propio framework puede generar (Playwright y Cypress lo hacen nativamente).



SÍNTESIS

Lo que hay que llevarse de hoy

1. **Selenium** es el más portable y el más verboso; requiere esperas explícitas.
2. **Playwright** resuelve la espera con auto-wait y trae depuración incluida.
3. **Cypress** corre dentro del navegador — rápido y simple, pero limitado a JS y un solo dominio.
4. **REST Assured** prueba la API directamente, sin interfaz — más rápido y estable.
5. Elegir framework depende del **contexto**: capa a probar, lenguaje del equipo, navegadores y madurez del proyecto.
6. Más automatización de UI **no es automáticamente mejor** — la pirámide de Cohn sigue aplicando.



INSTRUCCIONES

Taller de hoy: primer test E2E automatizado

2 horas, en sala de Zoom asignada — equipo fijo de Fábrica-Escuela

Sobre **The Internet (Herokuapp)**, escriban y ejecuten un primer test automatizado de extremo a extremo, con el framework que el equipo elija (Selenium, Playwright o Cypress) justificando la elección según los criterios de la sesión de hoy.

- El test debe cubrir un flujo completo (ej. login exitoso y login fallido) con al menos una aserción real sobre el resultado.
- **Evidencia de ejecución obligatoria:** captura de pantalla o reporte HTML del test corriendo en verde.
- Entregable: el código del test + la evidencia, en un repositorio o carpeta compartida por el equipo.

Herramienta: the-internet.herokuapp.com · **La entrega cierra al terminar las dos horas.** Recuerden: al enviar, cada integrante reporta en privado quién trabajó.



CIERRE

Para la próxima semana

Antes de la clase de la semana 14, hay que llegar con esto leído:

- Patrones de diseño para pruebas automatizadas: Page Object, Screenplay, data-driven, fixtures.

Seguimos con los equipos fijos de **Fábrica-Escuela**. El taller de la semana 14: refactorizar la suite de hoy aplicando Page Object, con comparativa antes/después en mantenibilidad.

